2

2장. 프로그램 정의

프로그램 목표 + 기능 요구사항 정의

무엇을 만들 것인가, 개발 시작 전 완전 정리

학습 내용

1. TaskFlow MVP 미션 + 페르소나 + 기능 5종 명확화
2. 화면 4개, DB 4테이블, API 18개 사전 설계
3. ACME 4요소로 OpenSpec 입력 준비

2-1. 왜 개발 전에 정의해야 하는가

스펙의 품질이 AI 산출물의 품질 상한을 결정함

무정의 던지기

한 마디씩 던지면 AI가 매번 다른 가정으로 구현. 결과 일관성 부족, 통합 시 충돌.

부분 정의

기능만 던지고 비기능은 생략하면 보안, 성능, 응답 시간이 AI 임의 결정으로 들어감.

완전 정의

미션 + 페르소나 + 기능 + 비기능 + 범위 외까지 정의. AI는 그 안에서만 결정.

2-2. TaskFlow MVP 프로그램 정의

1줄 미션 + 페르소나 3종 + 사용 시나리오

미션

소규모 팀이 칸반 + 실시간 채팅으로 업무 진행을 한 화면에서 추적함

페르소나

팀 리더	팀원	신규 합류자
3-5인 팀 운영, 진행 상황 파악, 우선순위 조정. 한 화면 대시보드 선호	내 태스크 빠른 확인, 칸반 드래그, 짧은 채팅으로 빠른 의사결정	초대코드로 가입, 기존 컨텍스트 빠른 파악, 채팅 이력 검색

사용 시나리오

1. 리더가 팀 생성 + 초대코드 발급 > 멤버 합류 > 칸반에 태스크 추가 > 진행 변경 > 채팅으로 합의
2. 신규 합류자 가입 > 초대코드로 팀 진입 > 칸반 + 채팅 이력 1분 안에 파악
3. PC에서 작성, 모바일에서 진행 확인. 자동배포된 URL로 어디서나 접근

2-3. 기능 요구사항 5종, 구현 범위

Day 2 마무리 시점까지 완성할 기능. 이 5종이 OpenSpec propose의 What이 됨

1	**인증**	회원가입, 로그인, JWT 발급, 비밀번호 해시
2	**팀**	팀 생성, 초대코드 발급, 합류, 멤버 목록
3	**칸반 태스크**	TODO/DOING/DONE 3컬럼, 추가, 상태 이동, 삭제
4	**채팅**	팀 단위 채팅 송수신, 폴링 5초, 발신자 표시
5	**배포**	Vercel 배포 MCP로 FE+BE 한 줄 배포 + Vercel Storage Neon

2-4. 범위 외 명시, 이번에 안 하는 것

AI에 범위 외도 명확히 알려야 임의 구현 안 함. proposal.md의 Out of Scope 섹션

Out of Scope, 이번에 안 하는 것 7종

[X]	**알림**	이메일/SMS/푸시 알림 없음. 채팅 폴링으로 대체
[X]	**파일 첨부**	이미지/파일 업로드 없음. 텍스트 채팅만
[X]	**검색**	전문 검색 없음. 단순 SELECT 조회만
[X]	**권한 세분화**	팀 admin/member 구분만. 페이지별 권한 없음
[X]	**다국어**	한글 UI만. 다국어 리소스 분리 없음
[X]	**WebSocket**	실시간 메시지 안 함. 5초 폴링으로 대체
[X]	**테스트 자동화**	pytest/jest 자동 테스트 없음. 수동 동작 확인만

2-5. 화면 사전 설계 4종, 와이어프레임

UI 사전 정의는 Vanilla JS + Tailwind 구현 시 AI 임의 디자인 방지

로그인 화면

- 상단 로고
- 이메일 입력
- 비밀번호 입력
- 로그인 버튼
- 회원가입 링크

팀 선택 화면

- 내 팀 목록
- 팀 만들기 버튼
- 초대코드 입력
- 초대코드 입력 + 합류 버튼

칸반 화면

- 3컬럼 TODO/DOING/DONE
- 각 컬럼 + 버튼
- 태스크 카드
- 드래그 상태 이동

채팅 화면

- 메시지 리스트
- 5초 자동 새로고침
- 입력창
- 발신자 + 시각 표시

2-6. DB 스키마 사전 정의, 4테이블 + 관계

로컬 SQLite + 배포 Neon (Vercel Storage). SQLAlchemy로 양쪽 호환 4테이블

users

```
id INTEGER PK
email TEXT UNIQUE
password_hash TEXT
created_at TIMESTAMP
```

teams

```
id INTEGER PK
name TEXT
invite_code TEXT UNIQUE
owner_id FK users
```

tasks

```
id INTEGER PK
team_id FK teams
title TEXT
status TEXT (TODO/DOING/DONE)
creator_id FK users
```

messages

```
id INTEGER PK
team_id FK teams
user_id FK users
content TEXT
created_at TIMESTAMP
```

2-7. API 엔드포인트 18개, 사전 정의

Auth 4 + Team 4 + Task 6 + Chat 4 = 18 엔드포인트. AI는 이 목록 그대로 구현만 함

Auth (4개)

```
POST /auth/signup
POST /auth/login
GET /auth/me
POST /auth/logout
```

Team (4개)

```
POST /teams (create)
GET /teams (my list)
POST /teams/join (invite_code)
GET /teams/{id}/members
```

Task (6개)

```
POST /teams/{id}/tasks
GET /teams/{id}/tasks
PUT /tasks/{id} (status)
DELETE /tasks/{id}
GET /tasks/{id}
PUT /tasks/{id} (title)
```

Chat (4개)

```
POST /teams/{id}/messages
GET /teams/{id}/messages?since=
GET /messages/{id}
DELETE /messages/{id}
```

2-8. ACME 4요소, 스펙 구조화 프레임워크

Assumptions, Constraints, Metrics, Examples 4축으로 OpenSpec 스펙을 빠짐없이 구조화

A — Assumptions 전제

역할

스펙이 성립하기 위해 깔고 가는 전제. 사용자는 로그인 상태, 입력은 UTF-8 같은 명시되지 않은 가정들

명시 안 하면

명시 안 하면 구현 검증 단계에서 충돌 발생. AI가 임의 가정으로 빈틈 채움

C — Constraints 제약

역할

반드시 지킬 한계 또는 규칙. 성능 응답 200ms 이내, 보안 PII 저장 금지, 특정 브라우저 지원 같은 것

명시 안 하면

할 수 없는 것 또는 반드시 해야 하는 것의 경계가 명확해야 함. 안 그러면 범위 밖 구현

M — Metrics 지표

역할

스펙 충족 여부를 객관적으로 판단할 수치 기준. 오류율 0.1% 미만, 사용자 80% 3클릭 내 완료 같은 측정 가능 형태

명시 안 하면

잘 동작 같은 애매한 표현을 숫자로 바꿔야 함. 안 그러면 검증 자체가 불가능

E — Examples 예시

역할

추상적 규칙을 구체 케이스로 보여줌. 정상 케이스, 엣지 케이스, 실패 케이스 함께 명시

명시 안 하면

해석 여지를 줄여줌. AI가 다른 해석으로 구현할 가능성 차단

2-8-1. ACME TaskFlow 실전 적용, 우리 프로그램에 어떻게 푸는가

4요소 개념을 TaskFlow MVP에 풍부하게 적용. 이게 OpenSpec propose의 핵심 입력

	요소	TaskFlow 적용 내용	핵심 의도
A	Assumptions	한국어 사용자만 / 팀당 5명 이내, 동시 50명 / 최신 Chrome+Safari / 초대코드 = 신뢰 / JWT localStorage	*전제 = 검증 안 함*
C	Constraints	무료 티어 한도 / bcrypt 해시 의무 / JWT 24h, 갱신 없음 / 메시지 1000자 / CORS 도메인 명시	*제약 = 반드시 지킴*
M	Metrics	API 100ms 이내 / 드래그 50ms 반응 / 배포 5분 이내 / 신규 합류 1분 파악 / 채팅 5초 폴링	*성공 측정 기준*
E	Examples	POST /auth/signup 201+JWT / 초대코드 ABCD-1234 / 상태 TODO,DOING,DONE / 시각 ISO 형식 / 에러 {code,msg}	*AI가 추측 못 함*

2-9. 비기능 요구사항, AI가 임의로 정하지 못하게

기술 스택, 보안, 성능 등 명시 안 하면 AI가 추측. 명시 = OpenSpec에 그대로 들어감

항목	내용	명시 이유
기술 스택	Backend FastAPI + 로컬 SQLite/배포 Neon + Frontend Vanilla JS + Tailwind	*AI가 Node.js 등 임의 선택 방지*
성능	API 응답 100ms 이내, 칸반 드래그 50ms 이내 반응	*체감 품질 기준선*
보안	JWT 발급 + bcrypt 비밀번호 해시 + CORS 허용 도메인 명시	*평문 저장 등 사고 차단*
배포	로컬 일체형(StaticFiles), 배포는 Vercel(FE+BE) + Vercel Storage Neon (Pooled 자동 주입)	*학습 단순 + 실전 표준 동시*
안정성	Neon 자동 백업 + point-in-time recovery (Free 플랜 기본). 로컬 SQLite는 git 제외	*데이터 손실 대비*
확장성	단일 서버 단일 DB, 마이크로서비스 분리 안 함	*MVP 범위 고정*
관측성	print 디버깅만, Sentry/로그 수집 없음	*Day 2 범위 외 명시*

2-10. 이 2장에서 정리한 모든 것이 OpenSpec propose 입력이 됨

미션 + 페르소나 + 기능 + DB + API + ACME + 비기능(기술 스택 포함) = AI가 받는 컨텍스트 전부

입력은 한 덩어리

AI에 '회의록 만들어줘' 한 마디가 아닌, 2장에서 정리한 PDF 2개(프로그램정의 + 스토리보드)를 한 번에 전달함

AI 임의 결정 없음

범위 외 명시 + 비기능(기술 스택) 명시 = AI가 추측할 여지가 사라짐

산출물 일관성

같은 입력 = 같은 산출물 구조. 팀원이 합류해도 컨텍스트 재설명 불필요

체크포인트

1. TaskFlow 미션과 페르소나 3종을 1분 안에 설명할 수 있음
2. 기능 5종과 범위 외 7종을 구분해서 말할 수 있음
3. DB 4테이블, API 18개, ACME 4요소, 기술 스택까지 사전에 정리 완료
4. 이 2장 전체가 OpenSpec propose 입력이 됨을 이해함